

# Emotiv Cortex Connection Reliability in MasterHub

## *Failure analysis and successful connection recovery*

Technical research report | 24 September 2026 | Version 1.0

### **Executive finding**

The MasterHub Cortex integration contained several software failure paths that could make connection recovery unreliable even when the local Cortex service was available. The principal defects were competing reconnect owners, credentials captured before later settings changes, and non-atomic credential persistence. The repair established one application lifecycle owner, loaded a fresh credential snapshot for each connection attempt, and committed settings through a quoted temporary file followed by atomic replacement.

The project owner reports that the connection succeeded after the file changes. This operational result is supported by regression checks of the repaired paths: the original focused run passed 50 tests, and the current expanded selection passed 70 tests during preparation of this report. These results support the software correction; they do not measure long-duration wireless uptime or prove which individual defect caused every earlier disconnect.

### **Purpose and audience**

This report explains the complete headset-to-MasterHub path for developers, project reviewers, and operators. It connects the observed failure conditions to code-level causes, identifies the exact repair scope, describes credential and prediction data flows, and provides a repeatable verification procedure.

### **Research approach**

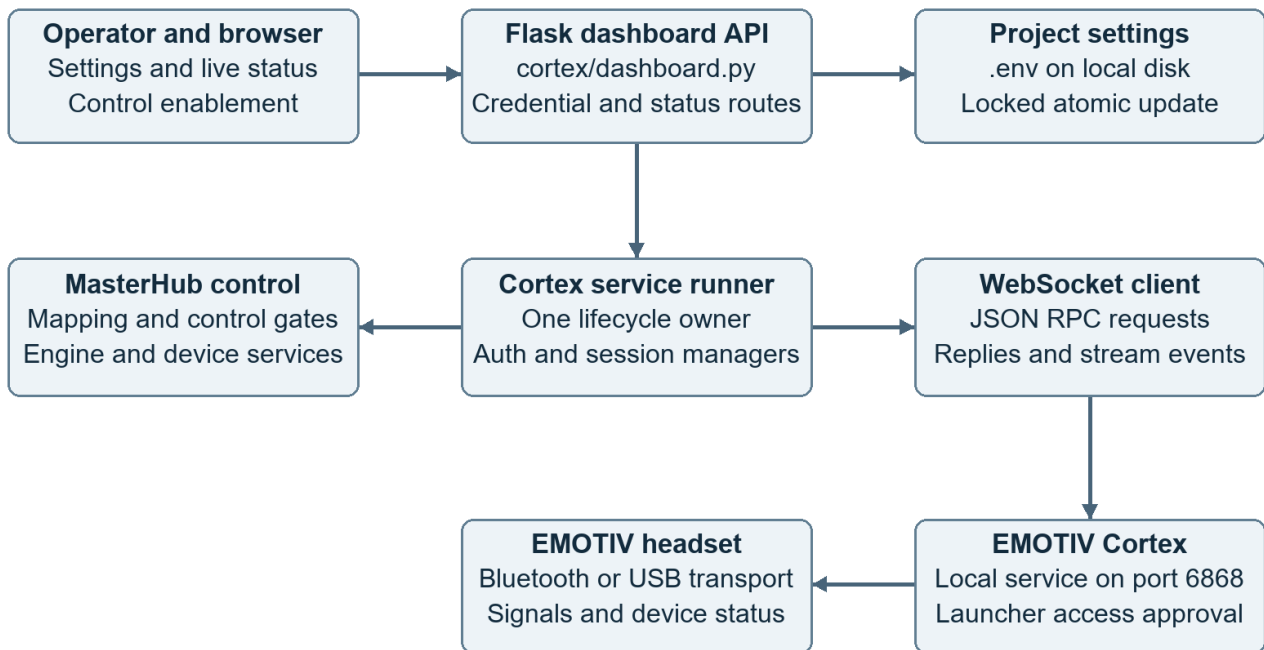
Evidence combines the earlier investigation and edit record, inspection of the current project files, deterministic automated tests, the project owner's success report, and EMOTIV's official Cortex API documentation. Application secrets, tokens, actual headset identifiers, and personal account data are intentionally excluded.

### **Reading guide**

Sections 1 and 2 explain the system and failure mechanisms. Sections 3 through 6 show the corrected ownership, connection sequence, and data flows. Sections 7 and 8 record the changes. Sections 9 through 12 cover runtime behavior, evidence, operation, and remaining research work. Section 13 provides the conclusion and references.

# 1 System architecture and connection boundaries

Cortex is a local EMOTIV service, separate from the MasterHub Flask application. A reachable Flask dashboard does not establish headset readiness, and an open Cortex WebSocket does not establish authorization or an active subscription. These layers must be checked independently. [R1, R3]



**Figure 1 Current system architecture with application control and configuration paths**

## Responsibility boundaries

The browser submits settings and displays status. The Flask blueprint persists settings and starts or stops the application-owned service. LiveRunner coordinates authorization, headset discovery, session creation, subscriptions, and recovery. CortexClient provides JSON-RPC request matching and unsolicited event dispatch over the secure WebSocket.

CortexAuth, HeadsetManager, SessionManager, StreamManager, and ProfileManager each own a protocol concern. CortexControl maps deliberate mental commands into the shared MasterHub engine. Device-specific services perform the final action and return acknowledgements. The radio or USB link between the headset and Cortex is an independent transport boundary.

## 2 Failure analysis and causal confidence

### Stale credential and endpoint values

Before the repair, authentication defaults and the WebSocket URL were captured from imported configuration values. Saving a new client ID or secret did not reliably replace every object that had already captured the old values. Updating only the active runner's credential fields also left future construction and URL selection vulnerable to stale defaults. This was a confirmed code defect; a complete saved settings snapshot is now loaded when creating default authentication and connection objects and before every runner retry.

### Competing connection recovery paths

The original runner could retry a lost connection while CortexClient independently scheduled a socket reconnect. A reconnect callback could additionally launch a recovery thread for authorization and subscription. These paths shared the same socket and managers. Their interleaving could close a newly opened socket, trigger more callbacks, or create overlapping session work. The race was identified from code ownership; a historical thread trace identifying its exact frequency was not captured.

### Credential persistence and test contamination

The old save helper rewrote .env directly and inserted unquoted values. A failed write could leave partial content, concurrent requests could lose updates, and dotenv syntax or interpolation could alter special characters. An existing credential endpoint test wrote to the real project settings. The initial investigation found a saved value matching a test-placeholder pattern. Test contamination is therefore a credible contributor, although the specific earlier writer was not independently traced.

### Protocol and lifecycle gaps

Headset discovery used headsetId where the documented queryHeadsets filter is id. A socket remaining open after stream cancellation was insufficient evidence of a healthy pipeline. Late events from a retired socket could also change the current connection state. The repair corrected the filter, handles Cortex warnings 0, 1, and 3, and rejects retired-socket open, close, and message callbacks. [R2, R4]

### What was not established as the root cause

The investigation did not establish a broken headset, a defective Bluetooth adapter, account entitlement failure, or poor signal quality as the original cause. Local ports 6868 and 5000 were reachable in the initial check, but that observation alone did not validate the saved credentials, Launcher approval, session state, or command samples.

### 3 Connection ownership before and after repair

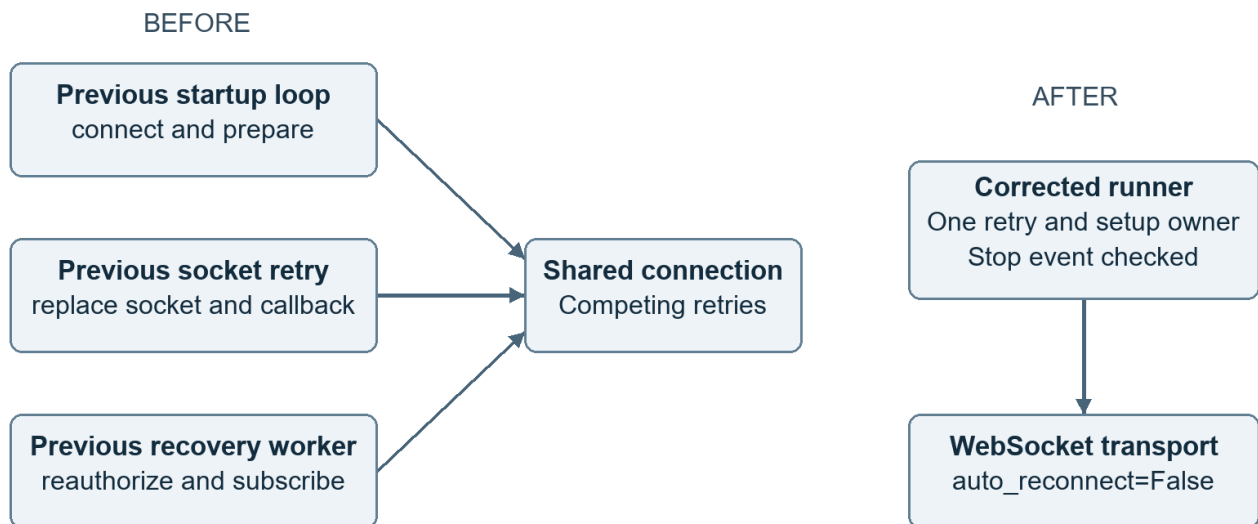


Figure 2 Competing recovery paths were replaced by one application lifecycle owner

#### One owner for complete recovery

The live integration constructs CortexClient with `auto_reconnect=False`. LiveRunner remains responsible for the complete sequence from socket connection through authorization and stream setup. The client still supports automatic reconnect for other uses, so this is an ownership decision for the live integration rather than removal of the generic transport capability.

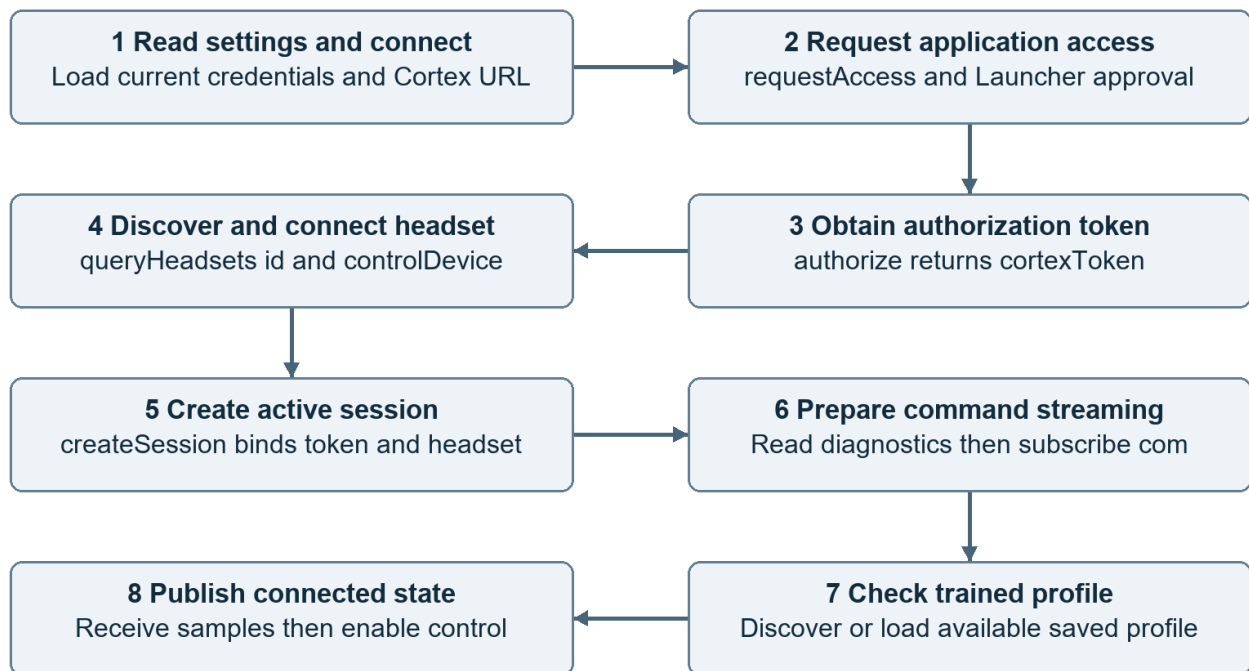
#### Event driven recovery requests

A settings save or relevant Cortex warning sets a reconnect event. It does not start another recovery worker. The runner observes the event, resets pending command processing, attempts to stop active motion, closes the old connection, clears stream callbacks and the token, reloads settings, and rebuilds the connection. Reconnection failure uses the same owner for the next attempt.

#### Service lifecycle coordination

A reentrant service lock serializes start and stop entry points. Stop sets the shared stop event and closes the socket; the worker completes its cleanup. Guards after connection and setup prevent a stopped runner from being reported as newly authorized. Blocking protocol calls may still take time to return, so stop is cooperative rather than an instantaneous thread termination.

## 4 Authorization and session establishment



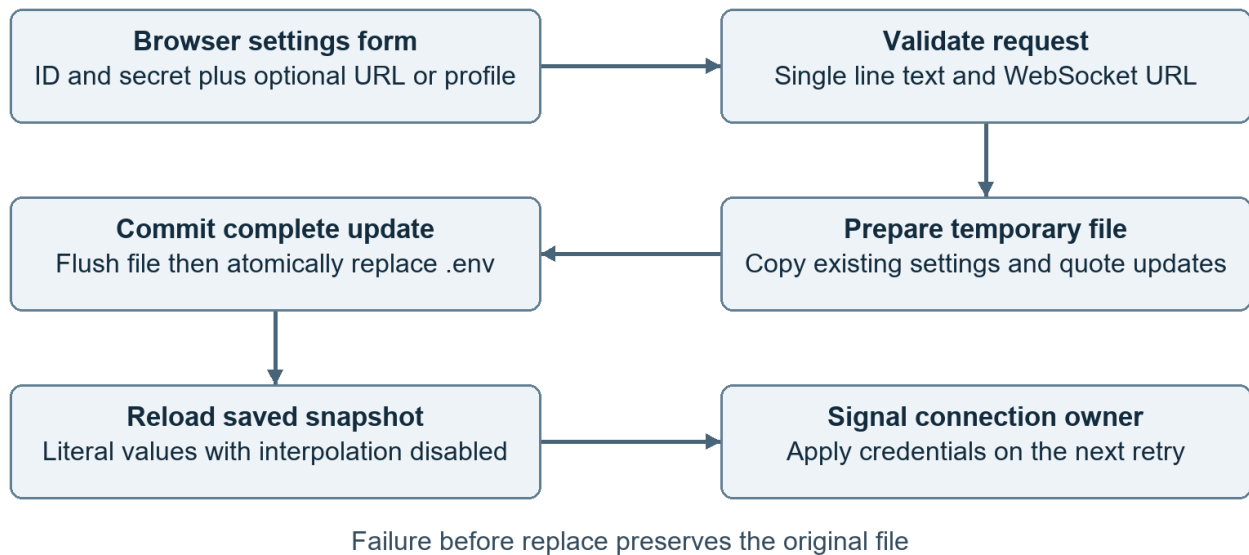
**Figure 3 Corrected connection sequence with the current profile preparation step**

requestAccess checks whether EMOTIV has approved the application. authorize obtains the token used by subsequent methods. Headset discovery and connection precede active session creation, after which the mental-command subscription can produce samples. EMOTIV documents the approval, discovery, and session stages separately. [R1, R3, R4]

A short device diagnostics subscription supplies available battery and signal information. A diagnostics-specific HeadsetError is logged without automatically treating it as failure of the mental-command path. Transport errors still require recovery. The same active session is used for diagnostics and command streaming.

Current code also discovers the trained profile, can load an available saved profile, and periodically refreshes profile state. These profile behaviors are relevant to the present architecture but were observed as additional workspace changes after the original connection repair. A usable connection and a suitable trained profile are separate prerequisites for command control.

## 5 Credential storage and configuration flow



**Figure 4 Credential save transaction and subsequent runtime application**

### Accurate persistence

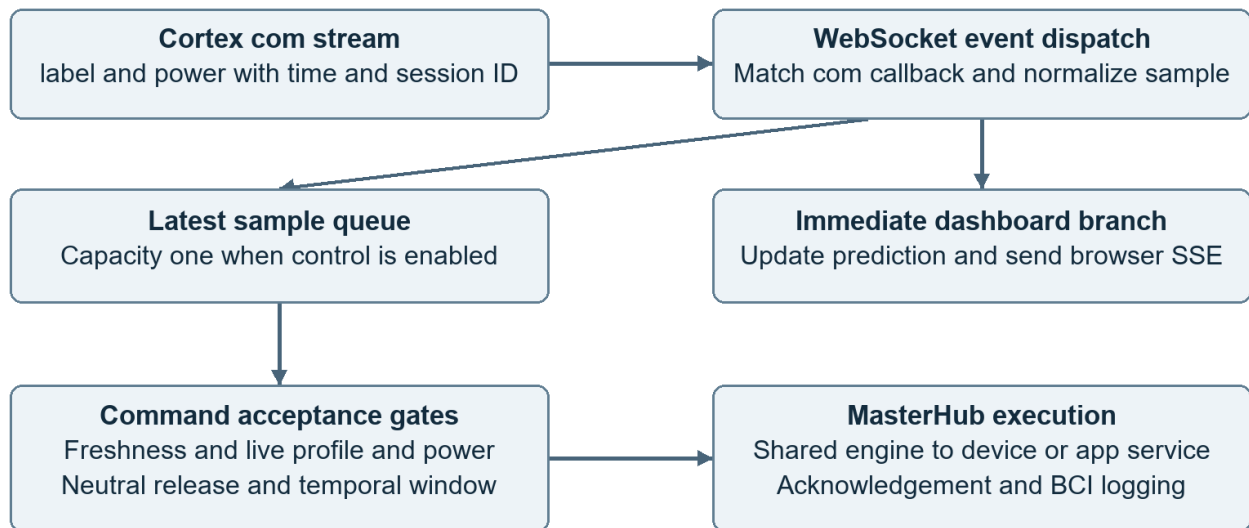
POST /cortex/credentials and its /cortex/config alias accept connection settings. Recognized fields must be single-line strings; CR, LF, and NUL are rejected. WebSocket URLs require ws or wss, a hostname, no embedded user information, and a valid parsed port. Values are trimmed before storage, so surrounding whitespace is intentionally normalized.

The writer holds the credential lock, copies the existing file into a same-directory temporary file, applies quoted dotenv updates, flushes the file, and calls `os.replace`. Readers using the same lock see a complete committed snapshot. Interpolation is disabled during loading, so a literal sequence such as `${HOME}` in a secret remains literal. The lock coordinates threads in one process; it is not a cross-process lock.

### Preservation and truthful status

An empty secret, or the exact mask returned for the current secret, preserves the stored secret. Other nonempty values, including legitimate asterisks, can be saved. GET returns a mask rather than the complete secret. A successful save means settings were committed; the response no longer claims that EMOTIV authorization or application registration has succeeded. The active runner is asked to reconnect with the saved snapshot.

## 6 Prediction and command data flow



**Figure 5 Mental-command data branches into live monitoring and gated command execution**

Cortex com messages carry a command label and power together with session and time metadata. The WebSocket client separates request replies by JSON-RPC ID from unsolicited stream events. The command callback normalizes each sample and publishes the dashboard prediction immediately. The browser receives updates through server-sent events rather than directly connecting to Cortex.

When control is enabled, the latest sample replaces the previous item in a queue of capacity one. The runner discards samples older than one second before command processing. CortexControl applies the current workflow, power threshold, neutral-release behavior, and temporal sequence window before calling the shared engine. Device handlers are isolated in regression tests; a passing test is not proof of physical actuator motion.

Acknowledgements and BCI logs describe execution results. Monitoring can continue without permitting commands. Recovery resets command state and requests a motion stop; safe physical stopping still depends on the device service accepting and carrying out that stop. The current command threshold and combination behavior are separate workspace features, not changes attributed to the original connection repair.

## 7 Production file change register

The following changes were made during the connection repair. Paths are relative to the MasterHub project root. Functions are more stable review locators than line numbers because the workspace has received subsequent edits.

| File and area                                        | Previous behavior                                                                                             | Repair and effect                                                                                                                                                                   |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| cortex/config.py<br>reload_credentials               | Reloaded dotenv into global environment; literal values could be interpolated.                                | Added a reentrant credential lock and dotenv_values with interpolation disabled. Saved values override inherited fallback values without rewriting unrelated process environment.   |
| cortex/auth.py<br>CortexAuth constructor             | Default AuthCredentials used values captured during import.                                                   | Default construction loads current settings and builds a new credential object. Explicitly supplied credentials remain supported.                                                   |
| cortex/cortex_client.py<br>constructor and callbacks | Default URL captured early; stale socket events were accepted; transport retry always active.                 | Resolve default URL dynamically; add auto_reconnect switch; make connected connect calls return; ignore retired open, close, and message events.                                    |
| cortex/cortex_client.py<br>logging and events        | Debug request messages included params; callback iteration used the live registry.                            | Remove request parameters from that debug message and iterate a registry snapshot. This reduces secret exposure in this logging path and mutation errors.                           |
| cortex/headset.py<br>discover                        | Sent headsetId as the discovery filter.                                                                       | Send the documented id parameter for targeted queryHeadsets requests.                                                                                                               |
| cortex/run_live.py<br>lifecycle                      | Runner retries and transport reconnect recovery could overlap.                                                | Disable transport auto-reconnect for this runner, use a reconnect event, rebuild from fresh settings, clear token and stream state, check stop, and react to warning codes 0, 1, 3. |
| cortex/service.py<br>start and stop                  | Service lifecycle entry points lacked a shared coordination lock; shutdown invoked runner cleanup externally. | Serialize start and stop, signal shutdown and close transport externally, and let the worker perform final runner cleanup.                                                          |

## 8 Settings and verification file change register

| File                                  | Change                                                                                                                                                   | Why it matters                                                                                                  |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| cortex/dashboard.py                   | Atomic quoted save; request validation; URL validation; common mask helper; reconnect signal; accurate save response.                                    | Preserves the complete settings file and avoids claiming external authorization from a local save.              |
| tests/conftest.py                     | Autouse pytest fixture redirects config and dashboard settings paths to a temporary file and restores modified globals.                                  | Credential API tests no longer write the real project .env when run under pytest.                               |
| tests/test_cortex_stability.py        | Added persistence, failed commit, partial failure, concurrent updates, input validation, mask preservation, stale socket, recovery, and discovery tests. | Exercises the defects directly with deterministic inputs and injected failures.                                 |
| tests/test_cortex_retry.py            | Changed the preservation test to submit the actual mask returned by GET.                                                                                 | Verifies the defined mask-preservation behavior instead of relying on an arbitrary string containing asterisks. |
| docs/CORTEX_CONNECTION_RELIABILITY.md | Added operating behavior, repair description, test command, and hardware validation steps.                                                               | Makes connection and persistence checks repeatable for future maintainers.                                      |

### Existing features retained

Session creation and conflict handling in session.py, generic stream management in stream.py, normalized prediction mapping, device routing, and BCI logging were used by the repaired lifecycle. They were not all newly implemented by this repair. The data-flow diagrams describe the complete system, including these existing components.

### Additional current workspace behavior

The current source includes profile refresh after connection, loading of a saved available profile, stricter profile ownership behavior, and expanded command timing and threshold tests. The current selected test count is 70 rather than the earlier 50. These additions are reported as current context; the repair record does not establish their authorship or attribute them to the preceding connection edit session.

### Scope of credential protection

Persistence is accurate and failure-resistant within the tested conditions, but .env remains a plaintext file. Masking an API response is not encryption. No operating-system credential vault, file access-control redesign, or cross-process transaction protocol was added.

## 9 Runtime recovery and timing behavior

| Condition                         | Current response                                                             | Important boundary                                                                              |
|-----------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Initial connection or setup error | Close the client and retry from fresh settings.                              | Runner startup retry delay defaults to 5 seconds, in addition to API operation time.            |
| WebSocket lost during operation   | Reset command state and rebuild through the single owner.                    | run_forever invokes recovery with a 2 second retry delay.                                       |
| Cortex warning 0 or 1             | Request recovery after subscription cancellation or automatic session close. | The WebSocket can remain open while the stream or session is unusable. [R2]                     |
| Cortex warning 3                  | Request recovery after EMOTIV logout.                                        | Successful recovery still requires the account to become usable. [R2]                           |
| Credential save                   | Set reconnect event after successful commit.                                 | A stopped service is not automatically started by saving settings.                              |
| Token soft age reached            | Runner rebuilds authorization and session.                                   | The six hour local soft age is an implementation policy, not a measured token expiry guarantee. |
| Explicit disconnect               | Set stop event, close transport, and allow worker cleanup.                   | Five second thread join does not forcibly terminate a still-running worker.                     |

### Transport defaults and limits

The source defaults are a 20 second WebSocket heartbeat interval, a 10 second heartbeat timeout, and a 10 second JSON-RPC request timeout. Headset connection has a separate wait window, and diagnostics wait briefly for a device sample. These configured bounds cannot be added together to claim a measured end-to-end recovery time; scheduling, radio discovery, account approval, and retries affect the actual duration.

The generic client still contains reconnect backoff and maximum-attempt settings. Because the live runner disables that client-level retry mechanism, those settings do not impose the live runner's retry policy. The application loop continues retrying until stopped. This distinction matters when tuning deployments.

## 10 Validation evidence and successful outcome

| Evidence                                          | Result                                                                        | Interpretation                                                                                                                  |
|---------------------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Original focused repair verification              | 50 passed with one initialization warning.                                    | Supported the original repair before subsequent workspace additions.                                                            |
| Latest selected verification on 24 September 2026 | 70 passed with one initialization warning in 2.04 seconds reported by pytest. | Confirms the selected current tests; duration is test execution time, not headset connection latency.                           |
| Earlier broader modernization run                 | 64 passed and 2 failed at that stage.                                         | Failures expected the old Cortex Classic page and successful profile loading while disconnected; not a clean full-suite result. |
| Local service availability during investigation   | TCP ports 6868 and 5000 accepted connections.                                 | Cortex and MasterHub were reachable; authorization and streaming were not established by this check.                            |
| Project owner follow-up                           | Connection reported successful after editing the files.                       | Operational confirmation from the user; no controlled duration, packet count, or restart sequence was supplied.                 |

### Latest selected test composition

The 70-test selection contains 15 stability cases, 4 retry and settings cases, 44 control cases, 1 modernization credential API case, and 6 BCI panel cases. The selection exercises credential round-tripping, file-write failure preservation, concurrent updates, stale socket rejection, recovery ownership, command gating, profile conditions, and live-panel behavior.

### Reproduction command

From the project root, run the following under the project Python environment. The pytest fixture isolates the credential file.

```
python -m pytest tests/test_cortex_stability.py tests/test_cortex_retry.py tests/test_cortex_control.py tests/test_modernization.py::test_cortex_credentials_api tests/test_bci_panel.py -q
```

### Causal conclusion

Code inspection establishes that the repaired failure paths existed. Regression tests establish expected behavior under controlled cases. The owner's report establishes that the edited system subsequently connected. Together these support the repair's effectiveness, while leaving the relative contribution of each defect and long-run reliability unquantified.

# 11 Operating procedure and troubleshooting

## Normal connection procedure

1. Start EMOTIV Launcher or the supported EMOTIV application and sign in. Make sure the headset is powered and visible through its intended transport. Start or restart MasterHub so that it loads the edited Python modules.
2. Open Connection Settings and save the real application client ID and secret. A blank secret preserves the stored value. Treat the saved indicator as persistence confirmation only. Approve the application in the EMOTIV prompt when requested.
3. Connect from MasterHub. Verify the authorization state, headset identity, active session, and changing mental-command samples. Check the trained profile independently. Enable control only after the intended profile and workflow are ready.
4. For persistence verification, stop and restart MasterHub and reconnect without re-entering credentials. For recovery verification, temporarily interrupt the headset link in a controlled setting, restore it, and check that a fresh usable session and samples return.

| Symptom                               | Likely layer to inspect                      | Next check                                                                                       |
|---------------------------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------|
| Settings saved but not authorized     | Application credentials or Launcher approval | Confirm genuine keys and approve access; inspect the current error without exposing secrets.     |
| WebSocket connected but no headset    | Discovery or physical transport              | Check headset power, pairing or dongle, selected ID, and EMOTIV visibility.                      |
| Active session but no useful commands | Subscription or trained profile              | Confirm com samples, profile load state, signal quality, and training.                           |
| Samples visible but no action         | Control gate or workflow                     | Check monitor mode, power threshold, neutral release, active menu, and required inputs.          |
| Repeated recovery or timeouts         | Transport, account, or lifecycle error       | Compare warning codes, last_error, retry state, and EMOTIV status.                               |
| Save returns an error                 | Filesystem or validation                     | Check allowed text, URL format, file permissions, and disk availability; retry after correction. |

Use `/cortex/state`, `/cortex/headset`, `/cortex/session`, dashboard events, and local logs together. Avoid interpreting a single green indicator as proof that every connection layer is healthy.

## 12 Remaining limitations and research plan

### Observed limits of the repair

The repair cannot correct poor wireless conditions, exhausted batteries, missing account approval, or an unavailable trained profile. A healthy socket also does not prove healthy samples. Current BCI control includes freshness gates, but a long-duration stream-health measurement was not part of the original work.

The settings lock is process-local. Two independent MasterHub processes or a simultaneous external editor can still compete over `.env`. Saved values intentionally take precedence over inherited values during reload. Atomic replacement protects the complete file transition under ordinary filesystem behavior, but it is not a general guarantee against every power-loss or storage failure.

The runtime accepts custom ws and wss endpoints, while certificate verification is disabled by default for the local self-signed Cortex setup. That default should not be treated as suitable for arbitrary remote endpoints. Plaintext secrets and partially masked responses remain security design considerations. Removing request parameters from one debug message does not audit every possible log path.

### Proposed hardware reliability experiment

Run a controlled set of at least 20 startup and recovery trials using the actual headset, followed by a two hour soak session. Record test start, connection established, authorization, active session, first sample, interruption, and recovered sample timestamps. Exercise a headset power cycle, Cortex service restart, application restart, credential change, and explicit stop during retry. These are proposed acceptance activities, not completed results.

Report connection success rate as successful trials divided by attempted trials. Report median and 95th-percentile recovery time from injected interruption to the first valid recovered sample. Record unexpected disconnects per hour, maximum sample gap, credential equality after restart, and whether queued actions were prevented after loss of control readiness. Establish acceptable thresholds before the experiment.

### Recommended follow-up engineering

Prefer one MasterHub owner process. If multi-process hosting is required, introduce shared configuration coordination and a dedicated connection service. Consider an operating-system secret store, stronger local API access controls, explicit recovery state reporting, and structured redacted telemetry. Add a hardware-assisted acceptance suite alongside the deterministic unit tests. Reconcile the older dashboard and disconnected-profile test expectations separately from connection recovery.

## 13 Conclusion and references

### Conclusion

The repaired design addresses the main software mechanisms that made Cortex integration unreliable: configuration snapshots were stale, settings writes were vulnerable to partial or altered persistence, and more than one path could own connection recovery. The resulting system uses one live lifecycle owner, reloads the saved credential pair and URL before reconnecting, rejects stale socket events, and responds to session and subscription warnings.

The project owner reports successful connection after the edits. The original 50 passing tests and the current 70 passing selected tests provide complementary software evidence. The next step for a quantified stability claim is repeatable hardware interruption testing and a measured soak run. The engineering outcome is a clearer, testable connection lifecycle with accurate settings persistence and a documented path to stronger reliability evidence.

### Official protocol references

[R1] EMOTIV Cortex API. Overview of API flow. Accessed 24 September 2026.

<https://emotiv.gitbook.io/cortex-api/overview-of-api-flow>

[R2] EMOTIV Cortex API. Warning Objects. Warning codes 0, 1, and 3. Accessed 24 September 2026.

<https://emotiv.gitbook.io/cortex-api/warning-objects>

[R3] EMOTIV Cortex API. Sessions. Session relationship and disconnection behavior. Accessed 24

September 2026. <https://emotiv.gitbook.io/cortex-api/session>

[R4] EMOTIV Cortex API. queryHeadsets. The id filter parameter. Accessed 24 September 2026.

<https://emotiv.gitbook.io/cortex-api/headset/queryheadsets>

### Implementation and evidence sources

[S1] Original investigation and edit record in this project task, including failure observations, source changes, test outputs, and the local service availability check.

[S2] Current source: cortex/config.py, auth.py, cortex\_client.py, headset.py, service.py, run\_live.py, dashboard.py, session.py, stream.py, profile.py, and control.py; static/bci-panel.js; associated tests and the connection reliability guide.

[S3] Latest selected pytest run during report preparation: 70 passed, one warning, 2.04 seconds. The warning identifies initialization of the modernization test suite; it is not a connection error.

[S4] Project owner follow-up stating that the connection succeeded after editing the files. This is recorded as user-reported operational evidence.